

Apache Iceberg

Spark v3.4 集成模块源码设计文档

基于 Apache Iceberg v1.9.2 源码深度分析

项目	Apache Iceberg - Spark v3.4 Integration
版本	v1.9.2
模块	spark/v3.4/spark + spark-extensions
规模	~594 文件 (503 Java + 91 Scala)
Spark 版本	Apache Spark 3.4.x
文档类型	源码架构与设计分析

目录

第一章 整体架构概览

第二章 核心流程图

第三章 核心时序图

第四章 核心类设计说明

第五章 关键设计模式总结

第一章 整体架构概览

1.1 模块结构与分层架构

Iceberg Spark v3.4 集成模块是 Apache Iceberg 与 Apache Spark 3.4.x 之间的桥梁层，负责将 Iceberg 的表格式能力（ACID 事务、Schema 演进、时间旅行、分区演进等）无缝暴露给 Spark SQL 引擎。该模块通过实现 Spark DataSource V2 API，使得用户可以通过标准 Spark SQL 语法操作 Iceberg 表。

层次	模块/包	职责	关键类
SQL 扩展层	spark-extensions	ANTLR 语法扩展、分析/优化规则注入	IcebergSparkSessionExtensions
Catalog 适配层	o.a.i.spark	Spark Catalog 到 Iceberg Catalog 桥接	SparkCatalog SparkSessionCatalog
数据源层	o.a.i.spark.source	DSv2 实现：读/写/表	SparkTable, SparkScan SparkWrite, SparkBatch
Filter/类型层	o.a.i.spark	Expression 转换	SparkV2Filters SparkSchemaUtil
数据读写层	o.a.i.spark.data	Parquet/ORC/Avro 格式读写	SparkParquetReaders SparkParquetWriters
向量化读取层	o.a.i.spark.data .vectorized	列式批量读取	VectorizedSpark ParquetReaders
Actions 层	o.a.i.spark.actions	表维护操作	SparkActions RewriteDataFiles
Procedures 层	o.a.i.spark.procedures	SQL CALL 命令	SparkProcedures BaseProcedure
Functions 层	o.a.i.spark.functions	内置 UDF	BucketFunction TruncateFunction

图 1-1: Spark v3.4 模块分层架构

1.2 与 Spark DataSource V2 API 的对接模型

Iceberg 通过实现 Spark DSv2 核心接口来完成集成，以下是关键接口对接关系：

Spark DSv2 接口	Iceberg 实现	功能
TableCatalog SupportsNamespaces	SparkCatalog	表/命名空间 CRUD
Table, SupportsRead SupportsWrite, SupportsDeleteV2	SparkTable	单表读写删除入口
ScanBuilder SupportsPushDown*	SparkScanBuilder	扫描计划构建 谓词/列/聚合/Limit下推
Scan, SupportsReport Statistics	SparkScan SparkBatchQueryScan	执行扫描 统计/批处理/流处理

WriteBuilder SupportsDynamic/Overwrite	SparkWriteBuilder	写入计划构建
Write, RequiresDistribution AndOrdering	SparkWrite	执行写入 分发/排序/分区
DeltaWrite	SparkPositionDeltaWrite	MoR 行级操作
DataSourceRegister	IcebergSource	数据源注册入口

图 1-2: Spark DSv2 接口与 Iceberg 实现对接

第二章 核心流程图

2.1 Spark SQL 查询 Iceberg 表全流程

步骤	组件	操作描述
1	Spark SQL Parser	解析 SQL，生成逻辑计划
2	SparkCatalog	loadTable(): 加载 Iceberg Table
3	SparkTable	newScanBuilder(): 创建 SparkScanBuilder
4	SparkScanBuilder	pushPredicates(): SparkV2Filters 转换谓词并下推
5	SparkScanBuilder	pruneColumns(): 列裁剪(含元数据列)
6	SparkScanBuilder	pushAggregation(): 聚合下推(COUNT/MIN/MAX)
7	SparkScanBuilder	pushLimit(): 行数限制下推
8	SparkScanBuilder	build() -> SparkBatchQueryScan
9	SparkBatchQueryScan	taskGroups(): scan.planFiles() 生成 FileScanTask
10	SparkBatch	planInputPartitions(): 分配任务给 Executor
11	Executor	RowDataReader 读取数据 + DeleteFilter 过滤

图 2-1: 查询全流程

2.2 Spark 写入 Iceberg 表全流程

步骤	组件	操作描述
1	SparkTable	newWriteBuilder(): 创建 SparkWriteBuilder
2	SparkWriteBuilder	校验/合并 Schema, 确定写入模式
3	SparkWriteBuilder	build() -> SparkWrite 匿名子类
4	SparkWrite.toBatch()	返回 BatchAppend/DynamicOverwrite/OverwriteByFilter/COW/Rewrite
5	WriterFactory	在 Executor 创建 PartitionedDataWriter/UnpartitionedDataWriter
6	DataWriter	write(): 计算 PartitionKey, 写入文件(RollingDataWriter控制大小)
7	DataWriter	commit() -> TaskCommit(DataFile[])
8	Driver	收集 TaskCommit, 调用 Iceberg API: newAppend/newOverwrite
9	commitOperation()	设置 appld/WAP/Branch, 执行 OCC commit()

图 2-2: 写入全流程

2.3 SparkCatalog 加载表流程

步骤	方法	操作描述
----	----	------

1	loadTable(ident)	入口: 调用 load(ident)
2	判断 PathIdentifier	路径标识: HadoopTables.load()
3	icebergCatalog.loadTable()	通过 Iceberg Catalog 加载(支持 CachingCatalog)
4	new SparkTable(table)	包装为 SparkTable
5	表名后缀解析	失败回退: namespace 作为表名, name 作为后缀
6a	at_timestamp_xxx	时间旅行: SnapshotUtil.snapshotIdAsOfTime()
6b	snapshot_id_xxx	快照定位: 直接使用 snapshotId
6c	branch_xxx	分支: copyWithBranch()
6d	tag_xxx	标签: 获取 tag 对应 snapshotId
6e	changelog	返回 SparkChangelogTable

图 2-3: SparkCatalog 加载表流程

2.4 Filter 下推三级分类

SparkScanBuilder.pushPredicates() 实现三级过滤分类：

分类	条件	处理
完全下推(A)	可转换 + 选择完整分区	仅 Iceberg 评估, Spark 不再过滤
部分下推(B)	可转换 + 不选完整分区	Iceberg 文件裁剪 + Spark 行级过滤
不可下推(C)	无法转换或绑定失败	完全由 Spark 评估
filterExpressions	A + B 集合	用于 Iceberg scan.filter()
postScanFilters	B + C 集合	返回 Spark 做后续过滤

图 2-4: Filter 下推分类

2.5 Actions 维护操作一览

Action	功能	核心流程
rewriteDataFiles	数据压缩	扫描->分组(BinPack/Sort/ZOrder)->并行重写->提交
rewriteManifests	Manifest重写	读取所有Manifest->Spark并行重写->替换
expireSnapshots	过期快照清理	确定过期快照->收集文件->Spark并行删除
deleteOrphanFiles	孤儿文件清理	列出实际文件-列出引用文件->差集->删除
rewritePositionDeletes	Delete文件重写	扫描Position Delete->合并重写->提交
computeTableStats	计算统计	ThetaSketch 计算 NDV->写入 Puffin 文件
migrateTable	表迁移	创建Iceberg表->导入文件->更新Catalog
snapshotTable	表快照	创建新表->导入源表文件->独立管理

图 2-5: Actions 维护操作

第三章 核心时序图

3.1 Spark BatchRead 时序

序号	调用方	→ 接收方	操作
1	Spark SQL	→ SparkCatalog	loadTable(ident)
2	SparkCatalog	→ IcebergCatalog	loadTable(TableIdentifier)
3	Spark SQL	→ SparkTable	newScanBuilder(options)
4	Spark SQL	→ SparkScanBuilder	pushPredicates/pruneColumns/pushAgg/pushLimit
5	SparkScanBuilder	→ SparkV2Filters	convert(predicate) 逐个转换
6	SparkScanBuilder	→ IcebergTable	newBatchScan().filter().project()
7	Spark SQL	→ SparkScanBuilder	build() -> SparkBatchQueryScan
8	SparkBatchQueryScan	→ IcebergScan	planTasks() 规划扫描任务组
9	SparkBatch	→ Executors	planInputPartitions() 分配任务
10	Executor	→ RowDataReader	read() + DeleteFilter 过滤
11	RowDataReader	→ Spark	return Iterator[InternalRow]

图 3-1: BatchRead 时序

3.2 Spark BatchWrite 时序

序号	调用方	→ 接收方	操作
1	Spark SQL	→ SparkTable	newWriteBuilder(info)
2	Spark SQL	→ SparkWriteBuilder	overwrite()/overwriteDynamic()
3	SparkWriteBuilder	→ Schema	validateOrMergeWriteSchema()
4	SparkWriteBuilder	→ Spark SQL	return SparkWrite (重写 toBatch/toStreaming)
5	Spark SQL	→ SparkWrite	toBatch() -> BatchAppend/DynamicOverwrite/...
6	WriterFactory	→ Executor	createWriter() -> Partitioned/UnpartitionedDataWriter
7	Executor	→ DataWriter	write(InternalRow) 逐行写入
8	DataWriter	→ RollingWriter	滚动写入, 文件达到 targetSize 自动切分
9	Executor	→ Driver	commit() -> TaskCommit(DataFile[])
10	Driver	→ IcebergTable	newAppend/newOverwrite().commit()
11	IcebergCore	→ MetaStore	OCC CAS 提交新 Snapshot

图 3-2: BatchWrite 时序

3.3 Merge-On-Read 行级操作时序

序号	调用方	→ 接收方	操作
1	SparkTable	→ Extensions	newRowLevelOperationBuilder()
2	Extensions	→ ScanBuilder	buildMergeOnReadScan() 记住 snapshotId
3	Extensions	→ DeltaWriteBuilder	build() -> SparkPositionDeltaWrite
4	DeltaWriterFactory	→ Executor	createWriter() -> PositionDeltaWriter
5	DeltaWriter	→ DVWriter	DELETE: 生成 DV/PositionDelete 文件
6	DeltaWriter	→ DataWriter+Del	UPDATE: 新数据文件+删除标记
7	DeltaWriter	→ DataWriter	INSERT: 仅写新数据文件
8	Executor	→ Driver	DeltaCommit(dataFiles, deleteFiles)
9	Driver	→ RowDelta	newRowDelta().addRows().addDeletes()
10	RowDelta	→ RowDelta	validateFromSnapshot() 冲突检测
11	RowDelta	→ IcebergCore	commit() 原子提交

图 3-3: MoR 行级操作时序

3.4 Structured Streaming 微批时序

序号	调用方	→ 接收方	操作
1	SparkScan	→ Spark SS	toMicroBatchStream(checkpointLoc)
2	Spark SS	→ MicroBatchStream	initialOffset() 获取初始偏移量
3	Spark SS	→ MicroBatchStream	latestOffset(start, readLimit)
4	MicroBatchStream	→ Table	currentSnapshot() 检查新快照
5	MicroBatchStream	→ MicroBatches	generateMicroBatch() 生成微批
6	Spark SS	→ MicroBatchStream	planInputPartitions(start, end)
7	Executor	→ RowDataReader	读取增量数据文件
8	Spark SS	→ StreamingWrite	commit(epochId, messages)
9	StreamingWrite	→ Table	newFastAppend/newOverwrite() 提交
10	Spark SS	→ Checkpoint	持久化偏移量

图 3-4: Streaming 微批时序

第四章 核心类设计说明

4.1 SparkCatalog - Catalog 适配层

SparkCatalog 继承 BaseCatalog，实现 Spark TableCatalog/SupportsNamespaces/ViewCatalog，将操作委托给 Iceberg Catalog。

类层次	SparkCatalog extends BaseCatalog (implements TableCatalog, SupportsNamespaces, ViewCatalog, Pr
核心字段	icebergCatalog, asNamespaceCatalog, asViewCatalog, cacheEnabled, tables(HadoopTables)
初始化	解析缓存配置 -> buildIcebergCatalog() -> 可选 CachingCatalog 包装 -> 识别 NamespaceSupport/ViewC
表加载	5种标识: PathIdentifier, 正常表名, at_timestamp_xxx, snapshot_id_xxx, branch_xxx/tag_xxx
时间旅行	loadTable(ident, version): 按版本号/引用名; loadTable(ident, timestamp): 微秒转毫秒
表创建	createTable->SparkTable; stageCreate/stageReplace->StagedSparkTable(Transaction)
Alter变更	分类: SetProperty/RemoveProperty/ColumnChange; 拦截 sort-order/identifier-fields; Transaction 原子提交

表 4-1: SparkCatalog 设计

4.2 SparkTable - 表抽象层

SparkTable 实现了读/写/删除/行级操作等多个 Spark 接口，是 Iceberg Table 在 Spark 中的表示。

实现接口	Table, SupportsRead, SupportsWrite, SupportsDeleteV2, SupportsRowLevelOperations, SupportsMetadata
核心字段	icebergTable, snapshotId(Long), branch(String), refreshEagerly(bool), isTableRewrite(bool)
Capabilities	BATCH_READ, BATCH_WRITE, MICRO_BATCH_READ, STREAMING_WRITE, OVERWRITE_BY_FILTER, OVERWRI
元数据列	_spec_id(INT), _partition(STRUCT), _file(String), _pos(LONG), _deleted(BOOL), _row_id(LONG), _last_upd
读入口	newScanBuilder() -> SparkScanBuilder (传入 branch/snapshotId/schema)
写入口	newWriteBuilder() -> SparkWriteBuilder; newRowLevelOperationBuilder()
元数据删除	canDeleteWhere(): 检查分区过滤+StrictMetrics; deleteWhere(): deleteFromRowFilter
不可变复制	copyWithSnapshotId(long) / copyWithBranch(String) - 时间旅行/分支副本

表 4-2: SparkTable 设计

4.3 SparkScanBuilder / SparkScan - 读取体系

采用 Builder 模式，SparkScanBuilder 构建扫描计划，SparkScan 及子类执行扫描。

类继承关系
SparkScan (abstract) └ SparkPartitioningAwareScan └ SparkBatchQueryScan (+ RuntimeV2Filtering) └ SparkCopyOnWriteScan └ SparkChangelogScan └ SparkLocalScan (聚合下推本地扫描)

图 4-1: Scan 类继承体系

组件	核心职责
SparkScanBuilder	pushPredicates() 三级分类; pruneColumns() 列裁剪; pushAggregation() 聚合下推; pushLimit(); build()
SparkScan	toBatch()->SparkBatch; toMicroBatchStream(); estimateStatistics(含NDV/CBO); 30+自定义Metrics
SparkBatchQueryScan	运行时过滤(SupportsRuntimeV2Filtering); filterAttributes()暴露分区字段; rewritableDeletes()
SparkCopyOnWriteScan	COW 行级操作专用, 记录受影响文件
SparkChangelogScan	CDC 变更日志扫描, 支持时间/快照范围
SparkLocalScan	聚合下推后本地扫描(无需读文件)

表 4-3: Scan 体系职责

4.4 SparkWrite / SparkWriteBuilder - 写入体系

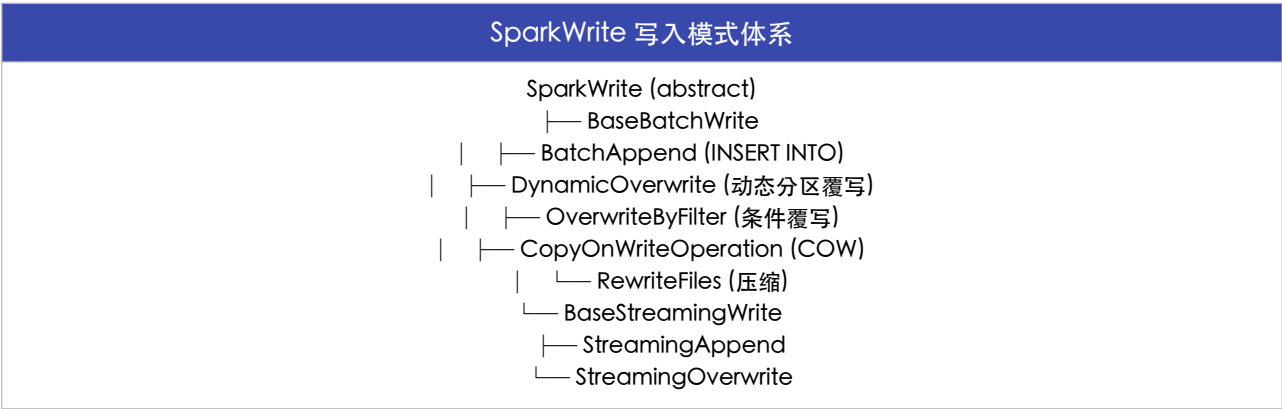


图 4-2: SparkWrite 模式体系

组件	核心设计
SparkWriteBuilder	确定模式 (append/overwrite/dynamic/cow); validateOrMergeWriteSchema(); 行血统 (Row Lineage) 注入
WriterFactory	序列化 Table (SerializableTableWithSize) -> Broadcast 广播; createWriter() 创建对应 Writer
Unpartitioned	RollingDataWriter: 达到 targetFileSize 自动切分新文件
Partitioned	计算 PartitionKey (InternalRowWrapper); fanout -> FanoutDataWriter; !fanout -> ClusteredDataWriter
BatchAppend	table.newAppend() -> appendFile() -> commitOperation()
DynamicOverwrite	table.newReplacePartitions(); 支持 SERIALIZABLE/SNAPSHOT 隔离级别冲突检测
CopyOnWrite	收集 overwrittenFiles + dangling DVs; newOverwrite().deleteFiles() + addFile(); 冲突检测
WAP支持	Write-Audit-Publish: STAGED_WAP_ID_PROP + stageOnly(); 暂存快照不影响当前指针
Streaming	幂等提交: queryId + epochId 去重; findLastCommittedEpochId() 检查历史

表 4-4: Write 体系设计

4.5 SparkPositionDeltaWrite - MoR 行级操作

实现接口	DeltaWrite, RequiresDistributionAndOrdering
命令支持	DELETE (仅 Delete 文件), UPDATE (Delete + 新数据), MERGE (Insert + Update + Delete 组合)
DV支持	启用 DV: PartitioningDVWriter 生成紧凑 DV 文件; 否则: ClusteredPositionDeleteWriter
Writer类型	BasePositionDeltaWriter (insert/update/delete), FanoutPositionOnlyDeleteWriter, PartitioningDVWriter
冲突检测	validateFromSnapshot (scanSnapshotId); SERIALIZABLE: noConflictingData + Deletes; SNAPSHOT: noConflictingData + Deletes
提交	Driver 收集 DeltaCommit (dataFiles, deleteFiles) -> RowDelta 原子提交

表 4-5: SparkPositionDeltaWrite 设计

4.6 SparkActions - 维护操作体系

类继承体系

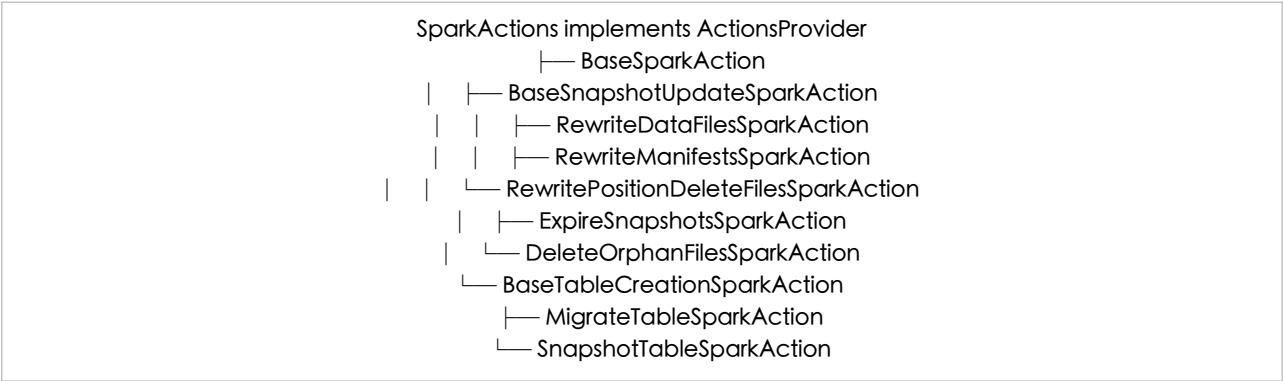


图 4-3: Actions 类继承体系

压缩策略	说明
BinPack(默认)	按文件大小合并小文件
Sort	按排序键重写, 优化数据跳过
ZOrder	Z-Order空间曲线排序, 优化多维查询
并行策略	MAX_CONCURRENT控制并发; PARTIAL_PROGRESS部分提交

表 4-6: 压缩策略

4.7 SparkProcedures - 存储过程体系

通过注册表模式管理 20 个存储过程, 用户可通过 CALL catalog.procedure_name(...) 调用。

存储过程	功能
rollback_to_snapshot	回滚到指定快照
rollback_to_timestamp	回滚到指定时间戳
set_current_snapshot	设置当前快照
cherrypick_snapshot	Cherry-pick快照
rewrite_data_files	数据文件压缩
rewrite_manifests	Manifest重写
remove_orphan_files	清理孤儿文件
expire_snapshots	过期快照清理
migrate	Hive迁移到Iceberg
snapshot	创建表快照
add_files	添加外部文件
register_table	注册表
publish_changes	发布WAP暂存变更
create_changelog_view	创建CDC视图
fast_forward	快进分支
rewrite_position_delete_files	重写Position Delete
compute_table_stats	计算表统计(NDV)
compute_partition_stats	计算分区统计
rewrite_table_path	重写表路径

表 4-7: 全部存储过程

4.8 SparkV2Filters - 谓词转换

类别	操作	转换结果
比较	=, <>, <, >, <=, >=	equal, notEqual, lessThan, greaterThan...
空值	IS NULL, IS NOT NULL	isNull, notNull
NaN	IS NaN, IS NOT NaN	isNaN, notNaN
集合	IN, NOT IN	in, notIn
字符串	STARTS_WITH	startsWith
逻辑	AND, OR, NOT	and, or, not

分区变换	years/months/days/hours bucket/truncate	对应Iceberg分区变换函数
------	--	-----------------

表 4-8: SparkV2Filters 转换操作

4.9 IcebergSparkSessionExtensions - SQL 扩展

通过 spark.sql.extensions 配置启用, 注入解析器/分析规则/优化规则/执行策略。

扩展类型	组件	功能
Parser	IcebergSparkSqlExtensions Parser	ANTLR语法: CALL, ALTER TABLE (PARTITION/BRANCH/TAG), VIEW
Analyzer	ResolveProcedures ResolveViews ResolveMergeIntoTable	解析存储过程/视图/MERGE引用 检查条件/对齐赋值
Rewrite	RewriteUpdateTable RewriteMergeIntoTable *ForRowLineage	重写UPDATE/MERGE为 Iceberg特定逻辑计划 支持行血统
Optimizer	ExtendedSimplify* ReplaceStaticInvoke RemoveRowLineageOutput	谓词简化/Null替换 静态调用替换 移除冗余行血统输出
Pre-CBO	RowLevelCommandScan* ExtendedV2Writes DynamicPruning	扫描下推/V2写入规则 动态裁剪/替换重写命令
Planner	ExtendedDataSourceV2 Strategy	逻辑计划->物理Exec节点 DDL/DML/Procedure映射

表 4-9: Extensions 全部扩展

第五章 关键设计模式总结

序号	设计模式	应用场景	关键实现
1	Builder 模式	扫描/写入构建	SparkScanBuilder/SparkWriteBuilder: 逐步设置后build()
2	模板方法	写入提交流程	SparkWrite.commitOperation()通用逻辑; 子类实现具体commit()
3	适配器模式	DSv2 API 对接	SparkCatalog/SparkTable/SparkV2Filters 三层适配
4	策略模式	压缩/写入/删除	BinPack/Sort/ZOrder; Clustered/Fanout; DV/PositionDelete
5	工厂模式	Reader/Writer	WriterFactory/SparkFileWriterFactory/SparkProcedures.newBuilder()
6	注册表模式	过程/函数管理	SparkProcedures(20个); SparkFunctions(10个); SPI注册
7	广播+序列化	Driver-Executor	SerializableTableWithSize; Broadcast[Table]; SparkInputPartition
8	OCC 并发控制	并发写入	SERIALIZABLE/SNAPSHOT隔离; validateFromSnapshot; conflictDetectionFilter
9	WAP 发布	安全发布	stageOnly()暂存; publish_changes发布; 不影响当前指针
10	Refinement	不可变扫描	copyWithSnapshotId/copyWithBranch; ScanBuilder每次build()独立
11	SPI 注册	数据源发现	IcebergSource: DataSourceRegister; META-INF/services SPI
12	委托模式	Catalog 分层	SparkCatalog 委托 icebergCatalog; SparkTable 委托 icebergTable

表 5-1: 关键设计模式总结

本文档基于 Apache Iceberg v1.9.2 源码分析生成，涵盖 Spark v3.4 集成模块的整体架构、核心流程图（查询/写入/Catalog/Filter/Actions）、核心时序图（BatchRead/BatchWrite/MoR/Streaming）、9 大核心类设计说明以及 12 种关键设计模式。